VOLUME I · PRODUCTION RECORD

VESPERA
THE BESTIARY

ENGINEERING & CREATIVE PRODUCTION FIELD GUIDE

Methods, architecture, difficult situations, code paths, repairs, validation, and lessons from building an interactive dark-fantasy journal.

FINAL PROJECT EDITION · JULY 2026

1. Final project at a glance

Measure	Final result
Validated monster records	136
Bestiary categories	11
Full generated portraits	136
Derived catalogue thumbnails	136
Delivered fieldcraft files	34
Signature illustrations	3
Static routes generated by Next.js	144
Database, CMS, authentication, runtime API	0
Production model	Static export on Vercel

Core technology

- Next.js 15 App Router and React 19
- TypeScript and Zod
- Tailwind CSS plus a large handcrafted global visual system
- Framer Motion for component and portrait movement
- GSAP with ScrollTrigger for reading-section reveals
- Lenis for smooth scrolling
- HTML Audio for supplied music
- Web Audio API for procedural ambience and interaction effects
- Python, PyMuPDF, RapidOCR, ONNX Runtime, OpenCV, and Pillow for data and assets
- OpenAI Codex for engineering, research orchestration, image workflow, testing, optimization, documentation, and deployment

The architectural principle was simple: expensive work happens before deployment. The browser receives static HTML, CSS, JavaScript, JSON-derived content, and optimized images.

2. Repository map

```
TEXT

app/
  page.tsx                          Landing route
  bestiary/page.tsx                 Catalogue server route
  bestiary/[slug]/page.tsx          Static monster route generation
  globals.css                       Parchment, layout, motion, responsiveness

components/
  bestiary/journal-browser.tsx      Search, category filtering, monster cards
  bestiary/monster-detail.tsx       Full creature study and unique portrait motion
  experience/opening-sequence.tsx   Scroll/click/keyboard journal opening
  experience/soundscape.tsx         Music, page turn, and sword interaction audio
  experience/smooth-scroll.tsx      Lenis lifecycle and reduced-motion handling
  experience/ambient-layer.tsx      Fog, dust, and atmosphere
  experience/sigil.tsx              Category sigil system

data/
  <category>/<slug>.json            One validated record per monster
  manifest.json                     Counts, categories, page references
  _reports/validation.json          Validation and missing-information report
  _research/witcher-wiki.json       Cached reviewed research

lib/
  schema.ts                         Zod contract and category definitions
  bestiary.ts                       Only application data-access module

public/
  monsters/generated/               Full optimized creature portraits
  monsters/thumbs/                  Lightweight catalogue images
  icons/fieldcraft/                 Oils, Signs, bombs, potions, effects
  images/                           Medallion, journal art, twin swords
  audio-local/                      Local/deployment music, intentionally Git-ignored

scripts/
  parse_bestiary.py                 OCR, extraction, visual classification, validation
  enrich_bestiary.py                Reviewed research and conservative inference
  optimize_assets.py                Full WebP and thumbnail production
  split_icon_atlases.py             Atlas segmentation and alpha cleanup
  create_favicons.py                Favicon family from the medallion
```

The shorter architectural reference remains at `docs/ARCHITECTURE.md`. This guide is the detailed companion.

3. Phase one: treating the PDF as data

The actual difficulty

The source was a 160-page illustrated field journal, not a clean text database. The useful content lived inside rendered pages. A naïve text extractor could not reliably discover titles, paragraphs, category transitions, icons, or portrait boundaries.

The parser therefore became a small document-understanding pipeline:

1. Render every PDF page at higher resolution.
2. Run OCR over the render.
3. preserve text, confidence, and bounding boxes.
4. Detect the page title using position, size, and confidence.
5. Correct repeatable OCR mistakes through an explicit review layer.
6. Infer the active category from known category transition entries.
7. Separate title and body lines.
8. Rebuild paragraphs from spatially sorted OCR lines.
9. Detect known Signs, bombs, and oils in the recovered prose.
10. Write one normalized JSON record per creature.
11. Produce a manifest and validation report.

Implementation: `scripts/parse_bestiary.py`

Dependencies: `requirements-parser.txt`

Local source location: `the-witcher-3-bestiary.pdf`

The PDF is intentionally excluded by `.gitignore`, but the parser and generated JSON are committed. That makes the website reproducible without distributing the original 64 MB source file.

Lazy-loading heavy parser dependencies

The parser does not import OCR libraries for a lightweight validation-only run. The heavy stack is loaded only when needed:

```python
def require_parser_dependencies() -> None:
    global Image, RapidOCR, cv2, fitz, np
    try:
        import cv2 as cv2_module
        import fitz as fitz_module
        import numpy as numpy_module
        from PIL import Image as pillow_image
        from rapidocr import RapidOCR as rapid_ocr
    except ImportError as exc:
        raise SystemExit(
            "Install parser dependencies from requirements-parser.txt"
        ) from exc
```

This matters because `npm run data:validate` should remain usable even when the machine is not configured for OCR.

OCR correction without silently changing facts

OCR consistently confused letters used by the journal font, especially `V`, `U`, and `Y`. The solution was not an uncontrolled spell checker. It was a small, reviewable correction map:

```python
OCR_FIXES = {
    "WOLUES": "WOLVES",
    "SILUER": "SILVER",
    "WYUERN": "WYVERN",
    "UAMPIRE": "VAMPIRE",
    "GRAUEHAG": "GRAVE HAG",
}
```

Title-specific corrections are stored separately in `TITLE_CORRECTIONS`. This distinction makes the parser auditable: a correction repairs a visibly present title; it does not invent a new monster fact.

Why OCR caching mattered

OCR over 160 high-resolution pages is slow. Each page result was cached under `.cache/ocr/`. The cache stored lines, confidence values, and boxes. Parser logic could then be adjusted repeatedly without paying the OCR cost again.

The cache is deliberately Git-ignored because it is reproducible and bulky.

Useful commands:

```bash
python -m pip install --target .parser-vendor -r requirements-parser.txt
npm run data:extract
npm run data:assets
npm run data:validate
```

4. From imperfect text to a trustworthy schema

Every monster uses the same contract. The schema lives in `lib/schema.ts` and is enforced with Zod during the Next.js build.

Important design decisions:

- Arrays are always arrays, even when empty.
- `swordType` and `dangerLevel` may be `null`.
- Every tactical fact records its provenance.
- Encounter information records whether it comes from a game, book, both, or a carefully marked inference.
- Source page, OCR confidence, and embedded-image count remain attached.

Simplified schema excerpt:

```ts
export const monsterSchema = z.object({
  schemaVersion: z.literal(1),
  slug: z.string(),
  name: z.string(),
  category: categorySchema,
  description: z.string(),
  story: z.string(),
  lore: z.string(),
  weaknesses: z.array(z.string()),
  oils: z.array(z.string()),
  bombs: z.array(z.string()),
  signs: z.array(z.string()),
  swordType: z.string().nullable(),
  habitat: z.array(z.string()),
  loot: z.array(z.string()),
  locations: z.array(z.string()),
  dangerLevel: z.string().nullable(),
  factSources: z.object({/* source for each tactical field */}),
  encounter: z.object({/* medium, title, location, note, URL */}),
  portrait: z.string(),
  gallery: z.array(z.string()),
  source: z.object({/* PDF page and OCR metadata */}),
});
```

The complete implementation is `lib/schema.ts`.

Validation strategy

`validate_records()` in `scripts/parse_bestiary.py` checks:

- category and directory agreement;

- schema version and required fields;
- duplicate names and slugs;
- invalid or missing portrait references;
- missing tactical information;
- record counts;
- normalization consistency.

Its committed output is `data/_reports/validation.json`. The final report marks the dataset valid, contains 136 records, and preserves a short list of genuinely missing lore or weakness fields rather than pretending every source was complete.

The “Unseen Elder” lesson

At one stage, searching for “Unseen Elder” returned no results. The problem was not the search component—it correctly searched the supplied monster summaries. The record itself was absent.

The fix required treating search failures as possible data-integrity failures:

1. inspect the generated manifest;
2. compare expected and actual entries;
3. add and validate `data/vampires/the-unseen-elder.json`;
4. generate and optimize its portrait;
5. rebuild the static route list.

The current record is:

```text
data/vampires/the-unseen-elder.json
```

Its full portrait and catalogue thumbnail are:

```text
public/monsters/generated/the-unseen-elder-hd.webp
public/monsters/thumbs/the-unseen-elder.webp
```

General lesson: when search finds nothing, verify the indexed data before rewriting the search algorithm.

---

5. Enrichment without hiding uncertainty

The original journal did not state every sword, habitat, loot table, location, or danger level. Showing “Not recorded in the source” everywhere made the experience feel like a transcription interface rather than a useful bestiary.

The enrichment pipeline in `scripts/enrich_bestiary.py` introduced a hierarchy:

1. retain explicit journal facts;
2. apply reviewed game reference data when available;
3. use book reference data for book-only creatures;
4. apply conservative category inference only as a final fallback;
5. label the source in `factSources`.

Category defaults are not presented as proven quotations. They are tactical inferences, such as silver swords for most supernatural categories or Specter Oil and Yrden for specters.

Representative pattern:

```python
CATEGORY_DEFAULTS["specters"] = {
    "swordType": "Silver",
    "oil": "Specter Oil",
    "bombs": ["Moon Dust"],
    "signs": ["Yrden"],
    "habitat": ["Haunted fields, graveyards, ruins, and sites of violent death"],
    "danger": "High",
}
```

Research cache: `data/_research/witcher-wiki.json`

Apply step:

```bash
npm run data:research
npm run data:enrich
npm run data:validate
```

The application does not show technical provenance language in the main visual experience, but the JSON retains provenance for future auditing.

6. Image production: 136 creatures without a placeholder

The batch-of-five workflow

Creating every portrait in one uncontrolled pass would have made failures hard to detect. The chosen workflow processed five monsters at a time:

1. select five records;
2. inspect their category, anatomy, lore, and silhouette;
3. create transparent or removable-background studies;
4. review completeness—wings, tails, horns, hands, feet, and weapons;
5. reject or regenerate damaged silhouettes;
6. clean alpha edges;
7. integrate the batch into the detail pages;
8. create lightweight thumbnails;
9. continue with the next five.

The generated-image prompts were part of an interactive production session and were not committed as a script. A representative prompt pattern was:

```text
Create an original dark-fantasy field-study portrait of <monster>.
Full creature visible from extremity to extremity, anatomically coherent,
front three-quarter view, readable silhouette, grounded realistic materials,
subtle painterly game-concept finish, no text, no frame, no scenery,
transparent background, generous safe margin around wings/tail/horns.
```

The “generous safe margin” requirement became important after several flying creatures appeared cropped.

Generated versus delivered assets

Full portraits:

```text
public/monsters/generated/<slug>-hd.webp
```

Catalogue derivatives:

```text
public/monsters/thumbs/<slug>.webp
```

The project currently contains 136 of each.

Diagnosing cropped monster images

The first question was whether the image files were already cropped or CSS was cropping them. That distinction determines the fix.

The efficient diagnosis:

1. open the raw asset directly;
2. inspect alpha bounds and canvas bounds;
3. inspect CSS `object-fit`, `object-position`, overflow, and container aspect;
4. compare detail and thumbnail variants.

The durable CSS fix uses containment:

```css
.monster-portrait-image {
  object-fit: contain;
  object-position: center bottom;
}

.card-portrait > img {
  object-fit: contain;
}
```

Locations:

```text
app/globals.css
```

If an extremity is missing in the raw image, CSS cannot restore it; that asset must be regenerated or placed on a larger transparent canvas.

Why two image tiers fixed the catalogue freeze

Loading 136 full-resolution portraits on the catalogue page wasted bandwidth and decoding time. `scripts/optimize_assets.py` creates:

- detail WebP up to 960 × 1200 at quality 88;
- catalogue WebP up to 240 × 300 at quality 78.

The application chooses the correct tier in `lib/bestiary.ts`.

```ts
export const getMonsterThumbnail = cache((monster: Monster): string => {
  const thumbnailName = `${monster.slug}.webp`;
  return fs.existsSync(path.join(thumbnailRoot, thumbnailName))
    ? `/monsters/thumbs/${thumbnailName}`
    : getMonsterPortrait(monster);
});
```

The optimizer validates every output before optional source deletion:

```bash
python scripts/optimize_assets.py --prune-sources
```

7. Fieldcraft icons, medallion, and swords

The original oils and Signs used mismatched source crops and inconsistent backgrounds. They were rebuilt into one polished visual family:

```text
public/icons/fieldcraft/
```

The application maps recognized preparation names to those assets in `components/bestiary/monster-detail.tsx`.

```ts
function iconPath(value: string) {
  const slug = normalizeToSlug(value);
  if (!known.has(slug)) return null;
  return `/icons/fieldcraft/${slug}.webp`;
}
```

Unknown values fall back to semantic React Icons, so a missing custom image does not break the page.

Icon atlas method

Some icons were produced as atlases and then split. The repeatable implementation is `scripts/split_icon_atlases.py`. It:

- divides the atlas into known rows and cells;
- estimates the subject mask;
- finds a square crop around the visible object;
- preserves transparency;
- resizes to a consistent delivery size.

This solved problems like a bottle being clipped at the side of its card.

Signature assets

```text
public/images/bestiary-medallion.webp
public/images/twin-witcher-swords.webp
public/images/vespera-journal-hero.webp
```

The medallion replaced an abstract landing rune because a readable circular field-study composition better communicated “bestiary” before the first word appeared.

Favicons are derived from that medallion by `scripts/create_favicons.py`.

8. Static application architecture

One controlled data boundary

Only `lib/bestiary.ts` reads monster files. React components never scan the filesystem and never hardcode monster records.

```ts
export const getAllMonsters = cache((): Monster[] => {
  return categories
    .flatMap((category) => readAndParseCategory(category))
    .sort((a, b) => a.name.localeCompare(b.name));
});
```

Every file is parsed through `monsterSchema`. A malformed record fails the build instead of silently rendering partial UI.

Static route generation

`app/bestiary/[slug]/page.tsx` turns the dataset into routes:

```ts
export const dynamicParams = false;

export function generateStaticParams() {
  return getAllMonsters().map((monster) => ({ slug: monster.slug }));
}
```

The same route computes previous and next creatures, which enabled continuous reading controls rather than the vague “Consult another entry” link.

Compact catalogue summaries

The catalogue does not receive the full object for every creature. The server route builds a summary containing only:

- slug;
- name;
- category;
- short story;
- thumbnail;
- prebuilt lowercase search text.

Implementation: `app/bestiary/page.tsx`

This reduced serialized page data and made filtering cheaper.

9. Building a journal rather than a wiki

Landing interaction

The opening sequence supports:

- click;
- Enter;
- downward wheel accumulation;
- upward touch gesture.

Implementation: `components/experience/opening-sequence.tsx`

The wheel threshold prevents a tiny accidental scroll from opening the book:

```ts
wheelDistance.current += event.deltaY;
if (wheelDistance.current >= 90) enter();
```

An `enteredRef` guard ensures click, wheel, and keyboard events cannot trigger duplicate navigation or duplicate page-turn sounds.

Sticky category journal

The category navigation is intentionally a journal margin rather than a navbar. Its desktop sticky behavior is defined in `app/globals.css`, while `components/bestiary/journal-browser.tsx` owns filtering and search.

The discarded Witcher-school medallion list taught an important interface lesson: thematic content is not automatically useful content. It forced an extra sidebar scroll and competed with the categories. Removing it improved both immersion and usability.

Larger click targets

Several labels looked like links but were hard to activate because only their glyphs accepted the click. The fix was to style the anchor itself as the interactive surface with padding and focus states.

General pattern:

```css
.detail-nav a,
.entry-pagination a {
  display: inline-flex;
  align-items: center;
  min-height: 44px;
  padding: 0.7rem 1rem;
}
```

This improves mouse, touch, and keyboard use without making the UI look like a modern button grid.

10. Monster pages and deterministic “unique” motion

Creating 136 hand-authored animation files would be difficult to maintain. Giving all monsters the same floating motion would look mechanical.

The chosen system combines:

- category temperament;
- a deterministic hash of the monster slug;
- small variations in amplitude, duration, direction, rotation, delay, and transform origin.

Implementation: `components/bestiary/monster-detail.tsx`

```ts
const hash = hashSlug(monster.slug);
const temperament = categoryMotion[monster.category];
const direction = hash & 1 ? 1 : -1;
const amplitude = temperament.amplitude * (0.82 + (hash % 37) / 100);
const duration = temperament.duration + ((hash >>> 8) % 330) / 100;
```

Why deterministic hashing is useful:

- every monster feels slightly different;
- the result remains stable between visits and builds;
- no hydration mismatch from `Math.random()`;
- no 136-entry animation configuration file.

Specters drift farther and glow more strongly. Ogroids move slowly. Insectoids are quicker. Vampires feel controlled but unnatural.

GSAP separately reveals reading sections once they reach the viewport. Framer Motion owns the persistent portrait behavior. Keeping those responsibilities separate reduced animation conflicts.

All motion checks `prefers-reduced-motion`.

11. Long harvest text and source contamination

Some loot fields accidentally contained long wiki fragments, markup, or bestiary instructions. Rendering everything made “Harvest” consume most of the page.

The UI now applies a defensive presentation limit:

```ts
const displayedValues =
  label === "Harvest"
    ? values
        .filter(
          (item) =>
            item.length <= 55 && !/===|\{\{|thumb|bestiary entry/i.test(item),
        )
        .slice(0, 4)
    : values;
```

Implementation: `components/bestiary/monster-detail.tsx`

The deeper lesson is that normalization belongs in the data pipeline, but presentation code should still protect the layout from an unexpectedly verbose record.

12. Audio architecture and its iterations

Implementation: `components/experience/soundscape.tsx`

Why two audio technologies are used

HTML Audio is best for long supplied music tracks:

- browser streaming and decoding;
- normal media playback;
- current time and duration;
- low implementation overhead.

Web Audio is best for generated interaction effects:

- procedural page paper;
- procedural fallback ambience;
- short sword pass;
- precise envelopes and filters.

The final playlist

The local playlist currently contains four tracks. The files are stored under `public/audio-local/`, ignored by Git, but included in direct Vercel deployments.

The player:

- chooses a random track on a fresh load;
- continues across landing and bestiary routes;
- advances automatically at the end;
- offers Play/Pause and Next;
- remains draggable;
- remembers its screen position;
- retries after the first gesture if autoplay is blocked.

Measuring and skipping leading silence

The tracks did not all begin at audible content. A temporary analysis environment used `miniaudio` to find the first 20 ms window above −50 dB. That environment lived in `.cache/audio-tools/` and was intentionally ignored.

Representative analysis:

```python
threshold = 10 ** (-50 / 20)
window = 441  # 20 ms at 22,050 Hz

for i in range(0, len(samples) - window, window):
    rms = sqrt(sum(x * x for x in samples[i:i + window]) / window)
    if rms >= threshold:
        audible_from = i / sample_rate
        break
```

Measured offsets committed in `soundscape.tsx`:

```ts
Kaer Morhen             2.44 seconds
Fields of Ard Skellig   0.74 seconds
Bad News Ahead          2.86 seconds
Geralt and Yen          0.02 seconds
```

The browser seeks on metadata load and verifies the offset again before play. This second check fixed an edge case where Play was pressed immediately after Next, before metadata had finished loading.

Autoplay is a request, not a guarantee

Modern browsers may block audible autoplay. The correct strategy is:

1. request playback on load;
2. do not treat rejection as fatal;
3. retry once on pointer, wheel, keyboard, or touch input;
4. expose a visible Play control.

No implementation can honestly guarantee audible autoplay in every browser.

Page turn

The page turn is a short generated noise buffer with:

- paper rasp;
- sweeping band-pass filter;
- playback-rate acceleration;
- low thump when the page lands.

The sound was increased after it disappeared beneath the soundtrack, but it remains short enough not to interrupt reading.

Card sound: why “Gwent-like” became a sword

The first card interaction combined an ink scratch and medallion chime. It was technically functional but felt like generic interface audio. A quieter first sword attempt still read as a scratch because the noise sweep was too narrow and its metallic tail resembled the old chime.

The final decision:

- no hover sound;
- one click-only broad air sweep;
- rapid low-to-high filter movement;
- an 85 ms descending steel edge;
- louder effect bus;
- no change to music volume.

The music remains at `0.34` while card effects use a separate Web Audio bus. Instrumented browser tests confirmed music stayed playing at the exact same volume before hover, after hover, and after selection.

General lesson: functional audio is not necessarily readable audio. Test what a sound communicates, not only whether sound nodes were created.

13. Performance: diagnosing “the click freezes”

The perceived freeze was not one bug. It was the combined cost of:

- a large 136-card DOM;
- oversized images;
- layout animation measurement;
- unnecessary complete monster payloads;
- continuous smooth-scroll work;
- development-server overhead.

The final response attacked each cost:

1. create catalogue-specific thumbnails;
2. send compact summaries from the server route;
3. remove expensive layout measurement from the complete grid;
4. apply `content-visibility: auto` to offscreen cards;
5. load full portraits only on detail routes;
6. pause Lenis when the document is hidden;
7. compare the production static preview, not only development mode.

Relevant paths:

```text
scripts/optimize_assets.py
lib/bestiary.ts
app/bestiary/page.tsx
components/bestiary/journal-browser.tsx
components/experience/smooth-scroll.tsx
app/globals.css
```

The catalogue rule:

```css
.monster-card-entry {
  content-visibility: auto;
  contain-intrinsic-size: 190px;
}
```

Performance lesson: interaction delays often come from work completed before the click—image decoding, layout, and animation bookkeeping—not from the route handler itself.

14. Testing methods used during the project

Static checks

```bash
npm run typecheck
npm run lint
npm run data:validate
npm run build
```

The production build is especially valuable because it:

- parses every JSON record;
- generates every static monster page;
- catches server/client boundary errors;
- verifies all 144 routes can be exported.

Browser-level audio instrumentation

Temporary Puppeteer tests lived in `.cache/browser-tools/` and `.cache/*.cjs`. They were ignored because they were diagnostic tooling, not production code.

A representative technique wrapped Web Audio constructors before the app ran:

```js
await page.evaluateOnNewDocument(() => {
  window.__audioNodes = { buffers: 0, oscillators: 0 };
  const NativeAudioContext = window.AudioContext;

  window.AudioContext = class extends NativeAudioContext {
    createBufferSource() {
      window.__audioNodes.buffers += 1;
      return super.createBufferSource();
    }
    createOscillator() {
      window.__audioNodes.oscillators += 1;
      return super.createOscillator();
    }
  };
});
```

The tests then verified:

- a random local track actually played;
- playback began after the measured silence offset;
- Next changed the source;
- Next while paused remained paused;

- music continued across route navigation;
- hover created no final card effect;
- click created the sword nodes;
- music volume remained exactly `0.34`;
- the public Vercel alias served the current implementation.

Why production verification mattered

An open browser tab can continue running an older JavaScript bundle after a new deployment. Deployment logs alone cannot distinguish a stale tab from a stale server.

The reliable check used:

- a fresh headless browser;
- a cache-busting query;
- the public production alias;
- observable behavior, not only HTTP 200.

This proved when the server was current and the remaining problem was sound design rather than deployment.

---

15. Deployment and repository hygiene

Static export

`next.config.ts` contains:

```ts
const nextConfig = {
  output: "export",
  trailingSlash: true,
  images: { unoptimized: true },
  poweredByHeader: false,
};
```

Next.js writes the portable site to `out/`.

Why audio is handled differently

`public/audio-local/` is ignored by Git. A Git-triggered deployment therefore cannot receive those local files. The project was deployed directly from the authorized local workspace so Vercel received the MP3s without committing them to the repository.

This creates an important operational distinction:

```text
git push             publishes source code
vercel deploy --prod publishes the local production payload, including audio
```

When final audio changed, both actions were needed.

Cache headers

`vercel.json` applies long-lived CDN caching to version-stable visual assets under `/monsters`, `/icons`, and `/images`, plus basic security headers.

Safe repository boundaries

Ignored local artifacts include:

```text
the-witcher-3-bestiary.pdf
public/audio-local/
.cache/
.parser-vendor/
.env*
.next/
out/
```

Generated JSON, optimized WebP assets, source code, documentation, and validation reports are committed.

16. Situations that changed the design

Situation: the landing page was black and only reacted after scrolling

The opening needed to communicate that scrolling was the intended gesture. The final design combined a visible cue, a medallion reveal, click support, keyboard support, and a wheel threshold. This preserved the scroll-to-open idea without trapping users who did not discover it.

Situation: category icons felt generic

Generic interface glyphs weakened the fiction. Category-specific sigils were introduced through `components/experience/sigil.tsx` and reused in the sidebar, cards, and detail portraits.

Situation: Witcher school medallions overcrowded the sidebar

The idea was thematic but made the category menu scroll independently. It was removed. Theme should reinforce the task, not compete with it.

Situation: monster portraits looked boxed and lifeless

Frames were removed, images used transparent containment, category-colored auras were introduced, and each slug received deterministic motion. The result feels integrated into the dark field study rather than pasted inside a card.

Situation: oil icons were clipped

The issue came from source framing and card containment. Atlas splitting, subject-aware square crops, and `object-fit: contain` solved it.

Situation: the project felt like a transcription

Technical source language was removed from the visible experience. Missing tactical fields were enriched with labeled game, book, or inference sources. The interface speaks like a Witcher journal; the JSON preserves audit details.

Situation: soundtrack routing was wrong

The first model used one track for landing and another for reading. The desired experience changed to a shuffled shared playlist. Replacing two competing audio elements with one active media element simplified route continuity and Next.

Situation: a sound existed but did not feel right

Ink, chime, and an early sword synthesis all passed technical tests. They still failed the artistic requirement. The system was simplified to a click-only, broader, louder sword pass. This is a valuable product lesson: automated tests can verify mechanics, but human judgment must verify meaning.

17. Reproducing the project

Application

```bash
npm install
npm run dev
```

Development URL:

```text
http://localhost:3000
```

Data pipeline

Place the source PDF at:

```text
the-witcher-3-bestiary.pdf
```

Install local parser tools:

```bash
python -m pip install --target .parser-vendor -r requirements-parser.txt
```

Run:

```bash
npm run data:extract
npm run data:assets
npm run data:validate
```

Optional reviewed enrichment:

```bash
npm run data:research
npm run data:enrich
npm run data:validate
```

Images

```bash
npm run assets:optimize
npm run assets:favicon
```

Use `--prune-sources` only after checking that every optimized output exists:

```bash
python scripts/optimize_assets.py --prune-sources
```

Final release check

```bash
npm run typecheck
npm run lint
npm run data:validate
npm run build
npm run preview
```

Static preview:

```text
http://localhost:3001
```

---

18. What should be preserved in future work

1. **Keep monster content outside components.** Components render schema data; they do not become a second database.
2. **Keep provenance in JSON.** The visual journal may be atmospheric, but the data should remain auditable.
3. **Maintain two portrait tiers.** Never make the catalogue load detail assets.
4. **Use deterministic motion.** Avoid random hydration differences.
5. **Respect reduced motion.** Atmosphere must not override accessibility.
6. **Treat autoplay as best effort.** Always retain visible controls.
7. **Test the exported build.** Development behavior is not production behavior.
8. **Validate the public alias.** A successful deployment log is not the same as a verified user experience.
9. **Do not confuse theme with clutter.** Every decorative system must support navigation, preparation, or reading.
10. **Let human taste overrule passing tests.** This mattered most for imagery, cropping, icon quality, and sound.

19. Quick path index

Question	Where to look
What fields must every monster have?	`lib/schema.ts`
Where are all monster files loaded?	`lib/bestiary.ts`
How are 136 routes generated?	`app/bestiary/[slug]/page.tsx`
How does search work?	`app/bestiary/page.tsx` , `components/bestiary/journal-browser.tsx`
How is the PDF parsed?	`scripts/parse_bestiary.py`
Where is the validation report?	`data/_reports/validation.json`
How is missing data enriched?	`scripts/enrich_bestiary.py`
How are portraits and thumbnails optimized?	`scripts/optimize_assets.py`
How are icon atlases split?	`scripts/split_icon_atlases.py`
How are favicons created?	`scripts/create_favicons.py`
How does the landing page open?	`components/experience/opening-sequence.tsx`
How does smooth scrolling work?	`components/experience/smooth-scroll.tsx`
How is unique monster motion produced?	`components/bestiary/monster-detail.tsx`
Where are fieldcraft icons mapped?	`components/bestiary/monster-detail.tsx`
How do music and effects work?	`components/experience/soundscape.tsx`
Where is the visual system?	`app/globals.css`
What controls static export?	`next.config.ts`
What controls Vercel headers?	`vercel.json`

Closing note

The most useful method in Vespera was not a library or framework. It was the loop:

```text
inspect → build → compare → reject → repair → validate → deploy → verify
```

The project became convincing because incomplete images were regenerated, cropped assets were diagnosed instead of hidden, missing monsters were treated as data failures, slow interactions were profiled across the whole page, and sounds were redesigned even after they technically worked.

That is the central lesson of the project: polish is not one final phase. Polish is the habit of refusing to let a small break in the fiction remain unexamined.